

# EV Truck Range Intelligence

AI Engineering — Case Study

---

**Delivery:** GitHub repo + README + one or two example agent conversations

## THE GOAL

NexDash is building the data foundation for an electric truck fleet. Over the coming years we'll gather large volumes of real telematics from our trucks, and we want a model at the centre of it that can reliably answer the question every dispatcher asks. This case study is the seed of that model. We are less interested in a perfect number today and more in how you build it, how you prove it, and how you'd grow it as real data arrives.

## THE PROBLEM

NexDash runs electric trucks (Mercedes eActros 600, ~600 kWh battery, ~500 km real-world range) across Germany. The single most important question in EV trucking is deceptively hard:

*"Will this truck make it to its next stop, and how much charge will it have left?"*

Energy use depends on distance, load, speed, terrain, weather and more, and getting it wrong means a 40-tonne truck stranded on the Autobahn. A model that answers this well underpins everything else we do: routing, charging, and scheduling.

## THE CASE

Two parts: build a model that answers the question and prove it works, then put a simple agent on top so a dispatcher can actually use it.

### Part 1 — Build the Model and Prove It (~70%)

*Goal: a data-driven model that predicts a truck's energy use for a route segment, with an honest account of how good it is.*

**Build it.** Create a data-driven model that predicts energy consumption (kWh) for a truck on a given route segment. The core inputs that matter:

- Distance
- Payload (0 to 22 tonnes)
- Speed
- Elevation / gradient
- Temperature

You're free to add other sensible features (e.g. wind, road type, HVAC load) if you can justify them. The model and feature choices are yours to make; we want to see you explain why your chosen approach fits this problem and how the raw inputs become your features. Sound reasoning matters more than complexity, and a simple model done well is exactly what we want.

There's no public energy dataset for 40-tonne EV trucks, so generating a small, justified synthetic dataset is a perfectly acceptable basis for training and testing. Explain what you generated and why it's realistic.

**Prove it.** Show us the model works, and where it doesn't.

- Hold out test data and report a real error metric (e.g. MAE in kWh or % range error).
- Show failure modes: when does it break? Cold weather? Steep grades? Heavy loads?

Honest evaluation is the most important thing in this case study. "Here's where it fails and why" is worth more than a perfect-looking number, and since you generated the data yourself, be candid about what your evaluation can and can't tell us. We're judging the honesty of your analysis, not chasing a target accuracy.

### Part 2 — Put a Simple Agent On Top (~30%)

*Goal: make the model usable through plain language.*

Wrap your model in a small LLM-powered agent so a dispatcher can ask a question in plain language and get a clear, reasoned answer, for example:

*"Truck's at 60% SOC with 12 tonnes, does it reach München in this weather?"*

The agent should call your energy model as a tool and explain its answer. Keep it simple; one clean, working interaction is enough. If you build the tool layer as an MCP server that's a bonus (it's our stack), but any clean tool interface works.

## WHAT TO DELIVER

- A repo we can clone and run, with clear setup instructions
- A README covering your approach, your evaluation results and failure analysis, your agent design, and a few sentences on how you'd grow this into a long-term system fed by real data (how the model retrains as data arrives, how you'd tell a new version is genuinely better before deploying it, and how you'd notice when it has drifted from reality)
- One or two example conversations with the agent (transcript or screen recording)

## GROUND RULES

- **Use any tools you want** (AI coding assistants, any LLM, any framework). Tell us what you used and why.
- **Mock or synthetic data is fine.** Real APIs are a nice touch if easy, never required.
- **CLI is perfectly fine.** A simple chat UI is a plus, not a requirement.
- **MCP server** is a bonus if you build the tool layer that way (it's our stack), but any clean tool interface works.

## WHAT WE'RE LOOKING FOR

| Criteria                    | What impresses us                                           |
|-----------------------------|-------------------------------------------------------------|
| Modeling & feature judgment | Thoughtful, clearly justified model and feature choices     |
| Honest evaluation           | Real metrics, failure analysis, knows the limits            |
| Long-term thinking          | A credible path from this seed to a system fed by real data |
| Code & communication        | Readable, runnable, and clearly explained                   |

We care about depth over breadth. A solid model with honest evaluation beats a flashy demo with no substance underneath.

*Good luck, and have fun with it. Electric trucks are cool.*